isep
École d'ingénieurs du numérique

Lab: Fault tolerance, Leader election, ZooKeeper-like System

II.3502 - Distributed Architecture and Programming - 2026/27

Objectives

- Understand consensus algorithms and leader election in distributed systems (Raft, ZAB).
- Implement a simplified ZooKeeper-like system featuring leader election, log replication, quorum commits, and recovery after leader failure.
- Apply a communication paradigm of your choice (sockets, RPC, RMI, or gRPC) to build a functioning distributed architecture.
- Build a small distributed game on top of the coordination layer to validate the system under realistic client interactions.

Modalities & Deliverables

This lab spans two sessions: the first is dedicated to development; the second to oral defence of your solution.

Evaluation breakdown:

Criterion	Weight
Deliverables (report + source code, submitted before defence)	40%
Oral defence	60%

Deliverables — to be submitted before the defence session:

- **Report** including:
  - Screenshots or screen recordings for all use-cases.
  - Brief analysis of your design choices and observed behaviour.
- **Source code** archive (all implementation files).
- **README** with run instructions: how to start nodes, simulate a leader crash, and run clients.



Defence: each student will be asked to demonstrate one of the three basic flows outlined in the Testing Scenarios section (chosen randomly on the day). The oral defence grade cannot be awarded based on the report alone — **live demonstration is required**. **Justified absence:** students unable to attend the defence session must contact the teacher individually to arrange an alternative demonstration date.

Groups: submission in groups of up to 3 students is authorised. Group composition must be communicated to the teacher during the first session.

Prerequisites

- Labs 1–2 (sockets, RPC/RMI, gRPC) concepts.
- Familiarity with Python or Java, threads/executors and basic I/O.
- Basic understanding of consensus (Raft/Paxos/ZAB) and quorum concepts.

Contents

1 Background **2**

2 ZooKeeper-like Game Application **5**

Background

Consensus in Distributed Systems

In a distributed system, independent nodes must **agree on a shared state** even when some nodes crash or messages are delayed. This is the **consensus problem**. A correct consensus algorithm guarantees:

Property	Meaning
Agreement	All non-faulty nodes decide the same value.
Validity	The decided value was proposed by some node.
Termination	Every non-faulty node eventually decides.

Failures that break consensus include **crash faults** (nodes stop), **network partitions** (messages lost), and **Byzantine faults** (nodes send wrong data). The **CAP theorem** states that during a partition a system must choose between consistency and availability.

Leader Election

The most practical approach to consensus is to elect one node as **leader** — it sequences all operations and replicates them to followers. If it fails, a new leader is elected.

Common strategies.

- **Bully algorithm:** node with the highest ID wins.
- **Ring election:** nodes pass tokens around a logical ring.
- **Raft:** a follower that times out becomes a candidate and wins a majority vote.
- **ZAB:** node with the highest `(epoch, seq)` wins; ties broken by node ID.

Heartbeat mechanism. The leader sends periodic **heartbeats** to followers. A follower that receives no heartbeat within a timeout assumes the leader has crashed and triggers a new election (Fig. 1).

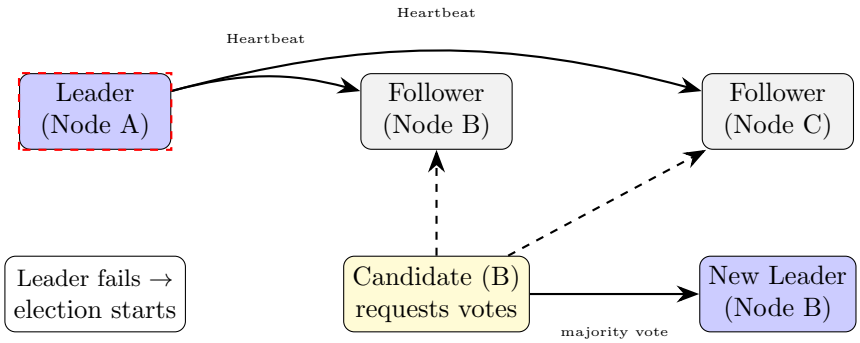


Figure 1: Heartbeat and leader election: on leader failure, a candidate collects a majority vote and becomes the new leader.

ZAB — ZooKeeper Atomic Broadcast

ZAB is the consensus protocol used by Apache ZooKeeper. Once a leader is elected it drives all updates through a **proposal** → **ACK** → **commit** cycle (Fig. 2):

1. Leader assigns a monotonically increasing transaction ID `(epoch, seq)` and sends a **Proposal** to all followers.
2. Each follower **logs** the proposal and replies with an **ACK**.
3. When the leader receives ACKs from a **quorum** (majority), it broadcasts **COMMIT**.
4. Followers apply the committed transaction in order; clients are notified.
5. On leader failure, the new leader **synchronises** uncommitted log entries before resuming.

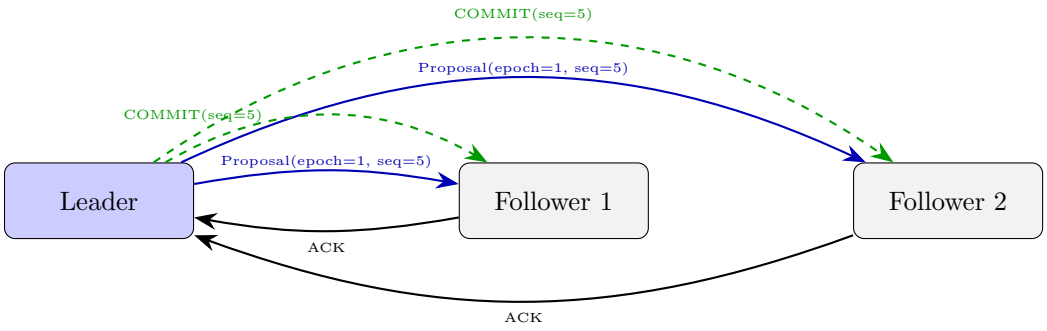


Figure 2: ZAB commit cycle: proposal (blue), acknowledgment, and commit broadcast (green).

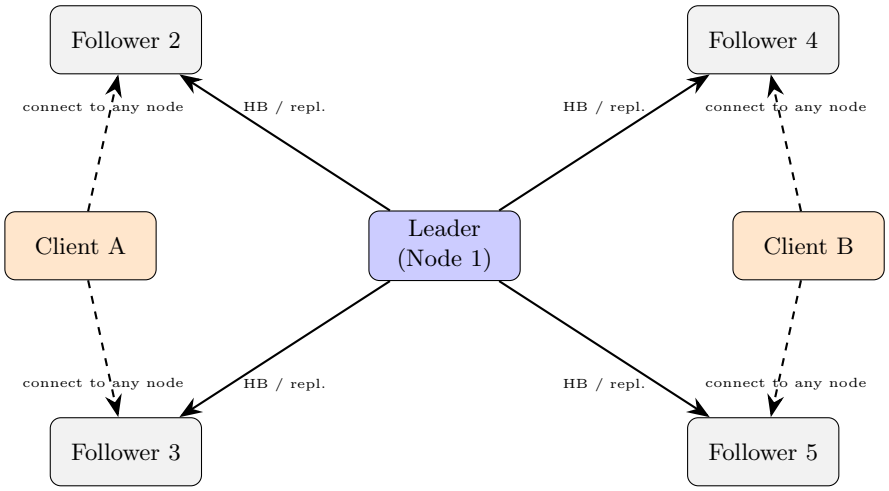


Figure 3: Lab topology: one leader, four followers, up to five clients. Clients connect to any node; followers forward writes to the leader.

System Architecture for This Lab

You will build a simplified ZooKeeper-like system with the topology shown in Fig. 3: one elected leader, four followers, and up to five clients. Clients may connect to *any* node; followers forward write and guess requests to the leader.



You may use **any communication paradigm** from the previous labs: raw TCP sockets, RPC (`xmlrpc`), RMI, or gRPC. Choose one and apply it consistently.

ZooKeeper-like Game Application

Overview

You will implement a distributed turn-based number game on top of the architecture introduced in Part 1. The game logic is intentionally simple so that you can focus on the distributed system mechanics: leader election, log replication, and fault recovery.

Game rules.

- The **first client** to connect sends a number (0–100) that becomes the **target**.
- Other clients submit **guesses**.
- The leader selects the **closest guess**, commits it as the new target, and a new round begins.
- Example: target = 50, guesses = {30, 62} → closest = 62 → new target = 62.

Node Roles

Role	Responsibilities
Leader	Assigns proposal tags `(epoch, seq)`, sends proposals to followers, collects ACKs, broadcasts COMMIT, sends heartbeats.
Follower	Logs proposals, sends ACKs, applies commits, forwards client write/guess requests to the leader, monitors heartbeat.
Client	Connects to any node, sends `Write(x)` (set target) or `Guess(x)`, reads current target.

Functional Requirements

Write — set target.

1. Client sends `Write(x)` to any node.
2. If received by a follower, it is forwarded to the leader; the follower does *not* log until a proposal arrives.
3. Leader assigns `(epoch, seq)` and sends **Proposal** to all followers.
4. Each follower **logs** the proposal with status `PROPOSED` and replies **ACK**.
5. On quorum ACKs (majority of nodes), leader broadcasts **COMMIT**.
6. Followers update log entry status to `COMMITTED` and make the new target visible to clients.

Guess — submit a guess.

1. Client sends `Guess(x)` to any node; followers forward to the leader.
2. Leader collects guesses for the current round and computes the closest value to the current target.
3. Leader treats the closest guess as a `Write` and follows the full proposal → ACK → COMMIT flow.

Leader failure and re-election.

1. Followers monitor the leader via periodic heartbeat. If no heartbeat is received within the timeout, a follower triggers an election.
2. Each candidate broadcasts its latest `(epoch, seq)`; the node with the highest `(epoch, seq)` wins. Ties are broken by **lowest node ID**.
3. The new leader **synchronises** uncommitted log entries across followers before resuming normal operation.
4. The new leader increments the epoch and resets seq to 0.

Log Format

Each node maintains a persistent local log. Every entry must record:

Field	Type	Description
`epoch`	int	Leader generation counter, incremented on each election.
`seq`	int	Proposal sequence number, incremented per proposal.
`op`	string	Operation type: `WRITE` or `GUESS`.
`value`	int	Proposed value.
`status`	string	`PROPOSED` or `COMMITTED`.

Message Protocol

Use a simple line-based or JSON format consistently across all nodes. Suggested message types:

Message	Fields
`PROPOSE`	`epoch, seq, op, value`
`ACK`	`epoch, seq, from`
`COMMIT`	`epoch, seq`
`HEARTBEAT`	`epoch, from`
`ELECTION`	`epoch, seq, from`
`FORWARD`	`op, value, from_client`

Implementation Hints

- **Networking:** Python: `socket` or `xmlrpc.server` or `gRPC`. Java: `ServerSocket`, `RMI`, or `gRPC`.
- **Concurrency:** Python: `threading` or `ThreadPoolExecutor`. Java: `ExecutorService`.
- **Persistent log:** Python: `sqlite3` or `pickle`. Java: local file I/O. Store one entry per line or row.
- **Quorum:** majority of total nodes, e.g. with 5 nodes quorum = 3.
- **Timeouts:** set explicit timeouts for ACK waiting and heartbeat monitoring; avoid indefinite blocking.
- **Local testing:** run each node as a separate process on `localhost` with distinct ports (e.g. 5001–5005).

Suggested Component Interfaces

- `NodeServer` — accepts incoming connections; dispatches `PROPOSE`, `ACK`, `COMMIT`, `HEARTBEAT`, `ELECTION`, `FORWARD` messages.
- `LeaderCoordinator` — assigns tags, sends proposals, waits for quorum ACKs, broadcasts COMMIT.
- `LogStore` — append / read / persist log entries; query latest committed entry.
- `ElectionManager` — monitors heartbeat timeout, triggers and participates in elections.
- `ClientApp` — connects to a node, sends `Write`/`Guess`, prints current target.

Testing Scenarios

All three scenarios must be demonstrable. Each carries equal weight in the defence.

1. **Normal write flow:** a client sets a target → leader proposes → quorum ACKs → COMMIT → all clients see the new target.
2. **Guessing flow:** multiple clients submit guesses → leader picks closest → commits new target → next round begins.
3. **Leader crash and recovery:** kill the leader process → followers detect missing heartbeat → new leader elected → logs synchronised → operations resume correctly.



Defence reminder: one of the three scenarios above will be chosen at random for your live demonstration. Ensure all three work before the defence session.

Resources

1. Apache ZooKeeper Overview: `https://zookeeper.apache.org/`
2. ZAB Protocol (original paper): `https://marcoserafini.github.io/papers/zab.pdf`
3. Raft Consensus Algorithm (interactive): `https://raft.github.io/`
4. Raft original paper — Ongaro & Ousterhout (2014): `https://raft.github.io/raft.pdf`
5. Paxos vs Raft — state transition comparison: `https://charap.co/reading-group-paxos-vs-raft-have-we-reacl`
6. CAP Theorem explained: `https://www.ibm.com/topics/cap-theorem`